



GuardianAI Gateway — Commandes Démo Complètes

Guide de démo du branchement du Raspberry Pi jusqu'à la démonstration finale.



ÉTAPE 0 — Avant de commencer (PC Fedora)

```
# Démarre les services Docker (PostgreSQL + Mosquitto + Grafana)
cd ~/9raya/Projects/gateway_project
docker compose up -d

# Vérifie que les 3 conteneurs tournent
docker ps --format "table {{.Names}}\t{{.Status}}"
```



Tu dois voir :

- gateway_postgres — Running
- gateway_mosquitto — Running
- gateway_grafana — Running



ÉTAPE 1 — Connexion au Raspberry Pi

```
# Branche le Pi (USB ou alimentation)
# Attends ~30 secondes qu'il démarre

# Connecte-toi en SSH
ssh pi@raspberrypi.local
# ou avec l'IP directe
ssh pi@10.17.245.6
# Mot de passe : ray+ray000
```



ÉTAPE 2 — Synchronisation du code (depuis le PC)

```
# Ouvre un NOUVEAU terminal sur le PC (pas le SSH)
rsync -av --exclude='venv' --exclude='__pycache__' --exclude='.git' \
~/9raya/Projects/gateway_project/ \
pi@10.17.245.6:~/gateway_project/
# Mot de passe : ray+ray000
```



ÉTAPE 3 — Démarrage des services sur le Pi

```
# Sur le Pi (terminal SSH)
cd ~/gateway_project
```

```

source venv/bin/activate

# Vérifie que les services réseau tournent
sudo systemctl status hostapd --no-pager | head -4
sudo systemctl status dnsmasq --no-pager | head -4

# IMPORTANT : Configure le routage NAT (accès Internet pour les appareils)
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward

# Ajoute la règle NAT si elle n'existe pas
sudo iptables -t nat -C POSTROUTING -o wlan0 -j MASQUERADE 2>/dev/null || \
sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE

# Ajoute les règles FORWARD pour le relais AP → Internet
sudo iptables -C FORWARD -i wlan0 -o uap0 -m state --state RELATED,ESTABLISHED -j
ACCEPT 2>/dev/null || \
sudo iptables -I FORWARD 1 -i wlan0 -o uap0 -m state --state RELATED,ESTABLISHED -j
ACCEPT

sudo iptables -C FORWARD -i uap0 -o wlan0 -j ACCEPT 2>/dev/null || \
sudo iptables -A FORWARD -i uap0 -o wlan0 -j ACCEPT

# Vérifie que tout est en place
sudo iptables -L FORWARD -v -n --line-numbers
sudo iptables -t nat -L POSTROUTING -v -n

# Sauvegarde pour les prochains redémarrages
sudo sh -c "iptables-save > /etc/iptables.rules"

```

✓ Ordre correct des règles FORWARD :

1. ACCEPT wlan0→uap0 ESTABLISHED (réponses Internet)
2. DOCKER-USER (Docker)
3. DOCKER-FORWARD (Docker)
4. DROP guardian rules... (blocages spécifiques)
5. ACCEPT uap0→wlan0 (tout le reste passe)

ÉTAPE 4 — Découverte des appareils

```

# Sur le Pi – connecte un téléphone au WiFi "GuardianGateway" d'abord
GATEWAY_ENV=pi python3 main_discovery.py

# Ou en mode daemon (continu toutes les 60 secondes)
GATEWAY_ENV=pi python3 main_discovery.py --daemon

```

✓ Tu dois voir les appareils avec MAC, IP, fabricant, type.

ÉTAPE 5 — Blocage d'appareil (iptables)

```
# Remplace le MAC par celui détecté à l'étape 4
MAC="a2:50:d5:8f:0b:04"

# 1. Ajout de la règle de blocage en base de données
python3 -c "from db import add_rule; add_rule('$MAC', 'block_device')"
```

```
# 2. Synchronisation de l'état (Applique le blocage dans iptables)
sudo /home/pi/gateway_project/venv/bin/python3 -c "
import os; os.environ['GATEWAY_ENV']='pi'
from main_rules import apply_rules
apply_rules(dry_run=False)
"
```

```
# Vérification
sudo iptables -L FORWARD -v -n --line-numbers | grep guardian
```

→ Sur le téléphone : **Google ne charge plus** ❌

```
# 1. Déblocage (on désactive toutes les règles pour l'exemple)
python3 -c "from db import get_connection; c = get_connection(); cur = c.cursor();
cur.execute('UPDATE rules SET active=FALSE'); c.commit()"
```

```
# 2. Synchronisation de l'état (Retire le blocage d'iptables)
sudo /home/pi/gateway_project/venv/bin/python3 -c "
import os; os.environ['GATEWAY_ENV']='pi'
from main_rules import apply_rules
apply_rules(dry_run=False)
"
```

→ Sur le téléphone : **Google fonctionne** ✅

ÉTAPE 6 — Blocage temporaire (schedule)

```
# Crée une règle pour les 5 prochaines minutes
GATEWAY_ENV=pi python3 -c "
from db import add_rule
from datetime import datetime, timedelta
now = datetime.now()
start = (now - timedelta(minutes=1)).strftime('%H:%M')
end = (now + timedelta(minutes=5)).strftime('%H:%M')
rule_id = add_rule('$MAC', 'schedule', start_time=start, end_time=end)
print(f'Règle #{rule_id} - blocage de {start} à {end}')
"
```

```
# Applique
sudo /home/pi/gateway_project/venv/bin/python3 -c "
import os; os.environ['GATEWAY_ENV']='pi'
from main_rules import apply_rules
apply_rules(dry_run=False)
"
```

```
"
```

```
# Vérification
```

```
sudo iptables -L FORWARD -v -n | grep guardian
```

ÉTAPE 7 — Blocage de sites (dnsmasq)

```
# 1. Ajout de la règle DNS en base de données
```

```
python3 -c "from db import add_rule; add_rule(None, 'block_category', 'reseaux_sociaux')"
```

```
# 2. Synchronisation de l'état (Applique dans dnsmasq)
```

```
sudo /home/pi/gateway_project/venv/bin/python3 -c "  
import os; os.environ['GATEWAY_ENV']='pi'  
from main_rules import apply_rules  
apply_rules(dry_run=False)  
"
```

```
# Vérifie le fichier de blocage
```

```
cat /etc/dnsmasq.d/guardian_blocks.conf
```

```
# Teste que TikTok est bloqué → doit retourner 0.0.0.0
```

```
host tiktok.com 127.0.0.1
```

→ Sur le téléphone : **TikTok/Insta bloqués**, Google OK 

```
# 1. Débloquent tout (désactive la règle)
```

```
python3 -c "from db import get_connection; c = get_connection(); cur = c.cursor();  
cur.execute('UPDATE rules SET active=FALSE'); c.commit()"
```

```
# 2. Resynchronise (vide le fichier dnsmasq et redémarre le service)
```

```
sudo /home/pi/gateway_project/venv/bin/python3 -c "  
import os; os.environ['GATEWAY_ENV']='pi'  
from main_rules import apply_rules  
apply_rules(dry_run=False)  
"
```

→ TikTok/Insta refunctionalise 



ÉTAPE 8 — MQTT temps réel (sur PC)

```
# Terminal 1 – écoute tous les events en temps réel
```

```
docker exec gateway_mosquitto mosquitto_sub -t "guardian/#" -v
```

```
# Terminal 2 – génère des events
```

```
cd ~/9raya/Projects/gateway_project
```

```
source venv/bin/activate
python3 main_discovery.py
```

✓ Messages JSON affichés en temps réel dans le terminal 1

ÉTAPE 9 — Dashboard Grafana

```
URL      : http://localhost:3000
Login    : admin
Password: guardian123
```

Panneau 1 — Appareils connectés (Table)

```
SELECT mac, ip, hostname, vendor, device_type, last_seen
FROM devices ORDER BY last_seen DESC
```

Panneau 2 — Types d'appareils (Pie chart)

```
SELECT device_type, COUNT(*) as total
FROM devices GROUP BY device_type ORDER BY total DESC
```

Panneau 3 — Règles actives (Table)

```
SELECT id, rule_type, mac, start_time, end_time, created_at
FROM rules WHERE active = true ORDER BY created_at DESC
```

ÉTAPE 10 — Tests automatisés (optionnel)

```
# Sur le Pi
cd ~/gateway_project
source venv/bin/activate
GATEWAY_ENV=pi pytest -v

# Sur le PC
cd ~/9raya/Projects/gateway_project
source venv/bin/activate
pytest -v
```

ÉTAPE 11 — Arrêt propre

```
# Sur le Pi – éteint proprement (évite corruption carte SD)
sudo shutdown -h now

# Sur le PC – arrête les conteneurs Docker
```

```
cd ~/9raya/Projects/gateway_project
docker compose down
```

Tableau des accès

Service	Adresse	Identifiants
Raspberry Pi SSH	ssh pi@raspberrypi.local	pi / ray+ray000
Grafana	http://localhost:3000	admin / guardian123
PostgreSQL	localhost:5433	gateway_admin / changeme123
MQTT Broker	localhost:1883	anonymous

Topics MQTT

Topic	Contenu
guardian/devices	Appareil détecté
guardian/rules	Règle appliquée/retirée
guardian/dns	Requête DNS catégorisée
guardian/alerts	Alerte